

審查系統設計文件 v3

系統目標

協助研究所招生委員會對大量推甄申請文件進行結構化分析，將非結構化的自傳與讀書計畫，轉化為可比較的能力輪廓，輔助人工審查決策。系統**不產生任何數值評分**，而是透過語意切割、標注與分群，讓審查者快速掌握每位候選人的能力分布與特色。

技術架構

元件	技術
後端	Node.js + TypeScript (ts-node / tsx)
LLM	Gemini API (文字切割、標注、sub-criteria 定義)
Embedding	本地模型 (@xenova/transformers, paraphrase-multilingual-MiniLM-L12-v2)
分群	K-Means + GMM (本地計算)
視覺化	UMAP + Plotly.js (靜態 HTML)

預定義 Criteria (評量維度)

Criteria 由校務會議預先定義，系統不自動探索或修改，共 4 個：

編號	名稱
C1	學術根基與跨域修課表現
C2	專題實作與技術應用能力
C3	問題解決與批判性思考
C4	專業傳遞與自我成長規劃

設計理由：Criteria 是評量「維度」而非互斥分類，同一個經歷可以在不同維度下被看見。例如一份研究專題可以展現 C2 (實作能力) 也可以展現 C3 (問題解決)。為避免標注結果過於分散，系統設計上**以單一 criteria 為主**，只有當 idea unit 確實同時展現兩個維度時才允許標注兩個。

Mock 資料生成

目的：真實推甄文件數量有限，且含有個人隱私，無法直接用於系統開發與測試。因此先以 LLM 生成具有多樣性的模擬文件，驗證 pipeline 的設計是否合理。

工具：mock_data/generate_mock.py (Python + Gemini API)

Phase 1: 建立學生屬性清單

從 `prompt/high_level_mock.txt` 讀取已有的學生屬性，以 `prompt/distribution.txt` 定義的統計分布規則，請 LLM 擴充到 100 筆。

每筆屬性包含：學校類型、主要研究領域、小方向標籤、跨領域亮點。

學校分布：頂大（台大/清大/交大/成大） 15%、中字輩國立 20%、其他國立 25%、私立 40%

領域分布：機器學習 20%、電腦視覺 15%、NLP 15%、資安 10%、計算機結構 10%，其他 30%

設計理由：屬性清單由 LLM 生成，確保多樣性，避免手動列舉時的盲點。分布規則對應台灣實際申請生態，使測試資料具有代表性。

Phase 2：依屬性生成推甄文件

對每位學生，Python 先決定性地（以學生編號為隨機種子）指派個人化參數，再交給 LLM 撰寫文件。

個人化參數（Python 決定，不讓 LLM 選）：

參數	選項
背景類型	技術型 / 社團型 / 打工型 / 創業型 / 跨域型
GPA	依學校類型分區間 (2.0–4.3)
文件格式	標準段落 / 條列式 / 散文式 / 混搭 / 過渡引導
文件長度	短 (500–800 字) / 中 (900–1500 字) / 長 (1600–2500 字)
撰寫品質	精心 / 一般 / 草率
姓名	從預設名字池隨機選，不讓 LLM 決定

設計理由：

- **Python 決定個人化屬性，而非 LLM：**若讓 LLM 自行決定風格與 GPA，容易趨中（都寫成精心、中等長度），失去測試的多樣性。Python 決定性指派能確保每次重跑結果相同，且覆蓋極端情況。
- **姓名由名字池隨機選：**LLM 選名字會導致常見名重複（如「志豪」出現太多次），名字池覆蓋性別、音節多樣性。
- **撰寫品質分層：**真實申請文件品質差異極大，草率文件（大量「獲益良多」、缺乏具體數字）對系統是重要的邊界測試。

輸出： `mock_data/mock_N.md` (Markdown 格式，含自傳與讀書計畫)

Pipeline 流程

步驟 0 — 文件前處理 (Preprocessing)

做了什麼：讀取原始文件，清除 OCR 雜訊後輸出乾淨文字。

清除項目包含：數字陣列（OCR 掃描後的數據殘留）、程式碼殘留行、孤立頁碼、連續點符號、多餘空白與換行。

為什麼這樣做：真實推甄文件多為掃描 PDF，OCR 轉換後常夾雜數值陣列、程式碼片段、頁碼等雜訊。若不清除，LLM 切割時可能將這些雜訊當成 idea unit 的一部分，降低後續標注品質。

指令： `npm run pipeline:mock` (內含 Step 0 + Step 1)

步驟 1 — Idea Unit 分割

做了什麼：將清洗後的整份文件，用 LLM 依語意切割成多個 **Idea Unit**，每個 Idea Unit 是一個語意完整、不可再分的能力展示單位。

切割規則：

- 每個 Idea Unit 需有獨立的語意，不依賴上下文即可理解
- 條列式活動紀錄：每條獨立為一個 Idea Unit
- 同時記錄文件來源（自傳 / 讀書計畫 / 加分文件 / 其他）

輸出： `output_mock/mock_N.json`，含 `idea_units[]` 陣列

為什麼這樣做：整份文件的一段文字通常同時包含多種能力描述，若以段落為單位標注，會混淆不同能力的訊號。切割成 Idea Unit 後，每個單位能精確對應一個（或少數幾個）能力維度，讓後續的量化分析更準確。

例如：

「在大二寒假，我擔任金融科技培訓課程講師，教導 200 位高中生 C++ 與 Python，同時也參與了 Deloitte 的產學合作，負責驗證訓練資料的真實性。」

這段文字包含「教學經驗」（C4）和「產學合作資料驗證」（C2），切割後各自成為獨立的 Idea Unit，才能分別標注。

指令： `npm run pipeline:mock`

步驟 2 — Idea Unit 標注

做了什麼：對每個 Idea Unit，LLM 執行三件事：

1. **Criteria 映射：**判斷這個 Idea Unit 主要展示哪個 criteria，預設只選 1 個。只有當內容中有明確文字同時描述第二個 criteria 的具體行為，且移除第二個後會明顯損失理解，才允許標注 2 個。
2. **Sub-criteria 命名：**對每個映射到的 criterion，以 4–10 個中文字自由命名一個 raw sub-criteria label，描述這個 Idea Unit 在該 criterion 下展現的具體能力面向。例：「跨域學科整合」、「ML 模型資料驗證」、「教學知識傳遞」。
3. **Hashtag 提取：**提取 0–5 個關鍵字（每個 2–8 字），分為技術/學術類（具體工具、技術、研究主題）與個人特質類（社團、競技、特定身份），要求讀者看到該 hashtag 能立刻認識這個學生的某個具體面向。

輸出欄位（寫入每個 IdeaUnit）：

- `criteria: CriterionId[]`
- `sub_criteria_map: Partial<Record<CriterionId, string>>`
- `hashtags: string[]`

批次處理：每 10 個 Idea Unit 為一批，送出一 API request，批次間隔 1.5 秒避免觸發 rate limit。

為什麼這樣做：

- **Sub-criteria 用自由命名而非預設清單：**每位候選人的具體能力面向不同，預設清單無法涵蓋所有情況。讓 LLM 自由命名後，再在 Step 3 收斂為標準詞彙，兼顧彈性與一致性。
- **Criteria 以單一為主：**早期設計允許 1-2 個 criteria，實測發現約 50% 的 Idea Unit 被標注兩個，其中大量是 C1 被泛用（只要是學術內容就貼 C1），失去鑑別度。改為預設只選 1 個後，雙 criteria 比例降至 7-16%，標注結果更能反映 Idea Unit 的主要能力方向。
- **Hashtag 強調具體識別度：**模糊的 hashtag（如「機器學習」、「積極」）對區分候選人沒有幫助，要求必須能讓讀者立刻聯想到該學生的具體特色。

指令：`npm run step2:mock`

步驟 3 — Sub-criteria 收斂

做了什麼：將步驟 2 中各候選人自由命名的 raw sub-criteria labels，以兩階段方式收斂為每個 criterion 底下的標準 sub-criteria 詞彙表。

Sub-criteria 的定義：位於 criteria（評量大方向）與 idea unit（個別具體描述）之間的中間層。

- 粒度：比 criteria 更具體，但不描述特定事件或個人
- 形式：3-8 字名詞短語，代表「一種展現方式或能力面向」，可跨候選人重複出現
- 例：「跨域學科整合」（不是「修了量子計算課」，也不是「有學術能力」）

兩階段流程（每個 criterion 各執行一次，共 4 次）：

Phase 1 — LLM 定義標準 sub-criteria

將該 criterion 底下所有候選人的 raw labels（附出現頻率）一次送給 LLM，要求歸納出 ≤ 10 個標準 sub-criteria，每個附上名稱（3-8 字）與說明（2-3 句），說明需具體描述哪類 idea unit 屬於此分類。

Phase 2 — Embedding 指派

使用本地 embedding 模型（`paraphrase-multilingual-MiniLM-L12-v2`，透過 `@xenova/transformers` 在 Node.js 本地執行），將每個 raw label 與每個標準 sub-criteria 的說明文字都轉為向量，以 cosine similarity 將每個 raw label 指派到最近的標準 sub-criteria。模型首次使用時自動下載（約 450MB），之後從本地快取載入。

輸出：`output_mock/standard_dictionary.json`，每個 criterion 底下若干標準 sub-criteria（含 `id`、`name`、`description`、`raw_labels[]`）

原子性：Phase 1 完成後立刻存檔（`raw_labels` 暫為空陣列），中途中斷重跑時自動偵測此狀態並跳過 Phase 1 直接進行 Phase 2。每個 criterion 的 Phase 2 完成後再次存檔。Embedding 結果快取至 `embeddings_cache.json`，不重複計算。

為什麼這樣做： 為了 candidate clustering, Criteria 粒度太粗，讓 LLM 知道某一群的學生有哪些 sub-criteria 比較多（比較著重在這邊）

- **為何要有 sub-criteria：**Criteria 粒度太粗，無法區分同樣標注為 C2 的候選人究竟是偏「資料分析」還是「系統開發」還是「硬體實作」。Sub-criteria 提供中間層的比較粒度，是後續特徵向量的構成單位。

- **為何改用 LLM 定義，而非原本的 k-means 聚類命名：**k-means 是無監督演算法，分群結果受隨機初始化的影響，產出的群組不保證有語意意義，命名需要額外 LLM call 補救。改為讓 LLM 直接看全部 raw labels 後定義標準，利用 LLM 的語意理解能力一步到位，品質更穩定。
- **為何 Phase 2 用 embedding 而非 LLM 做指派：**讓 LLM 對每個 raw label 逐一分類需要大量 API 呼叫，且輸出格式難以保證。Embedding cosine similarity 計算快、結果穩定，且因 Phase 1 的 description 已由 LLM 寫得具體，向量空間中的相似度能準確反映語意接近程度。[這個工作embedding就夠用了](#)
- **為何用本地 embedding 模型而非 Gemini Embedding API：**Gemini Embedding API 需要網路連線，實測在批次呼叫時偶有 `fetch failed` 網路錯誤導致整個 Step 3 中斷。本地模型完全離線執行，消除網路不穩定的單點失敗，且不產生額外 API 費用。選用 `paraphrase-multilingual-MiniLM-L12-v2` 的原因是原生支援中文（繁體），能正確理解中文 sub-criteria 名稱與說明的語意相似度。

指令：npm run step3:mock

步驟 4 — Sub-criteria 標準化回寫

做了什麼：讀取 `standard_dictionary.json`，對每個 candidate 的每個 idea unit，將 `sub_criteria_map` 中的 raw label 對應到標準 sub-criteria 名稱，寫入 `standard_sub_criteria_map`。

對應方式有兩種，視資料來源而定：

- **字串比對 (Step 4 預設)：**直接查 `standard_dictionary.json` 的 `raw_labels[]`，適用於同一批跑出來的 raw labels
- **Embedding 相似度 (relabel:embed)：**將 raw label embed 後與各 standard sub-criteria description 做 cosine similarity，適用於跨批次、raw label 不在字典內的情況（例如從舊資料複製過來的候選人）

輸出欄位（寫入每個 IdeaUnit）：`standard_sub_criteria_map: Partial<Record<CriterionId, string>>`

指令：npm run step4:mock (字串比對) 或 npm run relabel:embed (embedding 比對)

步驟 6 — 特徵向量建構

做了什麼：對每個 candidate，統計每個 standard sub-criteria 出現幾次（有幾個 idea unit 被指派到該 sub-criteria），組成一個 28 維計次向量 ($C1 \times 7 + C2 \times 8 + C3 \times 6 + C4 \times 7$)，做 L2 normalize 後存為 `feature_vector`。同時統計每個 criterion 底下有幾個 idea unit，存為 `radar_chart_data`（供雷達圖使用）。

名稱: [C11, C12,C21, C22,C31, C32,C41, C42,]

向量(文件中的sub-criteria個數): [35 ,37 ,]

為什麼這樣做：L2(類似標準化, 除以總個數): 有人因為文件比較長，所以每個sub-criteria的個數都很多，要讓文件長跟文件短的人，有得比較

- **為何用 sub-criteria 計次而非文字 embedding：**sub-criteria 計次向量代表候選人的「能力分布輪廓」，維度語意明確，可直接解釋哪個維度高代表什麼能力傾向。文字 embedding 雖然語意更豐富，但維度不可解釋，不利於後續向審查者展示分群理由。
- **為何 L2 normalize：**消除文件長短對距離的影響。寫了 20 個 idea unit 的候選人和只寫 8 個的候選人，若能力分布相似，normalize 後向量距離應接近。

指令：npm run step6:mock

步驟 7 — 分群與視覺化

兩個都試試，看哪個分數高，同樣是用silhouette算分數(0~1)

K-Means: 把相近的拉到同一群

GMM: 若總共有AB兩群，假設有一個點，屬於A群的機率是20%，屬於B群的機率是480%，則會被判定為B群

做了什麼：對所有候選人的 `feature_vector` 同時跑 K-Means 和 GMM，嘗試 K=2~6，以 silhouette score 挑選最佳演算法與 K 值。每群找出 medoid（群內平均距離最小的成員），將 `cluster_id` 與 `is_medoid` 寫回各 candidate 檔案，並輸出 `clusters.json`。另外跑 UMAP 將高維向量降至 2D，產生互動式 HTML 散點圖 (`umap.html`)。

UMAP方便看分布

輸出： `output_mock/clusters.json`、`output_mock/umap.html`

為什麼這樣做：

- **為何同時跑 K-Means 和 GMM：**K-Means 假設群為球形，GMM 允許橢圓形群，兩者各有優勢。以 silhouette score 自動選勝者，避免手動選擇偏誤。
- **為何找 medoid 而非 centroid：**centroid 是數學平均點，不對應任何真實候選人。medoid 是群內最具代表性的真實候選人，審查者可直接閱讀其文件來理解該群的特色。
- **為何用 UMAP 而非 PCA/t-SNE：**UMAP 保留全域結構（群間相對位置有意義），t-SNE 只保留局部結構；UMAP 速度也比 t-SNE 快。

Mock 資料的分群限制： 我們的資料: 在K=2,3,4,5中，K=2的分數最高，是0.166
0.7~1很棒，0.5~0.7還不錯，0.2~0.5勉強接受，0.2以下超爛

Mock 資料實測結果為 K-Means K=2、Silhouette=0.166，分群品質偏低，原因如下：

1. **Mock 資料設計為多樣性，而非群聚性：**mock 資料依照學校層級、研究領域、背景類型等分布刻意打散，候選人在特徵空間中呈連續分布而非離散群落，演算法找不到自然邊界，最終退化為 K=2（只能粗略切成兩半）。這三點是分數低的原因(藉口)，第一點最重要，(二三點會被質疑，可以不要講)
2. **Feature vector 稀疏：**28 維向量中每個候選人通常只有少數維度非零，稀疏向量之間的距離差異小，K-Means 難以找到清晰的群邊界。
3. **Sub-criteria 計次不反映內容語意：**兩位都有「ML 模型實作」idea unit 的候選人，特徵向量完全相同，即使研究方向截然不同，導致本應有差異的候選人被歸為同群。

真實推甄資料若有明顯能力族群（如純研究型 vs 系統實作型 vs 跨域應用型），分群品質預期會顯著改善。

指令： `npm run step7:mock`

步驟 8 — 分群命名

做了什麼：讀取 `clusters.json`，收集每個群內所有成員的 `standard_sub_criteria_map`，統計出各群 Top 5 sub-criteria，一次將所有群的統計資料送給 LLM，請 LLM 同時為所有群命名並描述各群特色。

為何一次送全部群：若每群獨立命名，LLM 不知道其他群長什麼樣，容易產出高度相似的名稱（如兩群都叫「技術實作型」）。一次送全部群讓 LLM 能在比較中找出各群的相對差異，命出互相區分的名稱。

LLM 輸出（每個群）：

- **name：**4-8 字中文群名，代表這群人的核心能力方向
- **description：**2-3 句自然語言描述，說明這群申請者的特色與輪廓，不提及統計數字

為何移除 hashtag：hashtag 高度個人化，在數十人的群內幾乎不重疊，最高頻的 hashtag 可能也只有 3~5 人有，不具群體代表性。Sub-criteria 才是跨候選人共用的結構化標籤，能真正反映群體傾向。

輸出：`cluster_name`、`cluster_description` 寫回 `clusters.json` 與每個成員的 candidate JSON

指令： `npm run step8:mock`

步驟 9 — 個人亮點標籤

做了什麼：對每個 candidate，收集所有 idea unit 的 `hashtags` 欄位並去重，將完整清單送給 LLM，請它從中挑選 3–4 個最能代表該申請者亮點的標籤，存為 `distinctive_hashtags`。

重點在挑選，而非生成：LLM 只能從 Step 2 已產出的 hashtag 裡選，不會憑空創造新標籤，確保輸出有原始文件依據。

挑選標準（prompt 明確規定）：

- 優先具體技術領域（「硬體安全」「FPGA 加速」優於「機器學習」）
- 特殊經歷（競賽名次、論文、業界實習）
- 排除通用詞、個人特質、學校科系名稱

每個候選人獨立一次 API 呼叫，呼叫間隔 1 秒。重跑安全：`distinctive_hashtags` 已存在者自動跳過。

輸出：每個 candidate JSON 新增 `distinctive_hashtags: string[]`（最多 4 個）

指令：`npm run step9:mock`

資料模型（步驟 0–9 相關欄位）

IdeaUnit

```
{
  id: string;
  candidate_id: string;
  section: '自傳' | '讀書計畫' | '加分文件' | '其他';
  content: string;
  // Step 2 填入:
  criteria: CriterionId[];
  sub_criteria_map: Partial<Record<CriterionId, string>>; // raw label
  hashtags: string[];
  // Step 4 填入:
  standard_sub_criteria_map?: Partial<Record<CriterionId, string>>; // 標準名稱
}
```

Candidate

```
{
  candidate_id: string;
  source_files: string[];
  cleaned_text?: string;
  idea_units: IdeaUnit[];
  // Step 6 填入:
  feature_vector?: number[]; // 28 維 L2-normalized sub-
  criteria 計次向量
  radar_chart_data?: Record<CriterionId, number>; // 各 criterion 的 idea unit 數
}
```

量

```
// Step 7 填入:
cluster_id?: number;
is_medoid?: boolean;
// Step 8 填入:
cluster_name?: string;
cluster_description?: string;
// Step 9 填入:
distinctive_hashtags?: string[];
}
```

StandardDictionary (standard_dictionary.json)

```
{
  [criterionId: string]: Array<{
    id: string;           // e.g. "C1_S1"
    name: string;         // 標準 sub-criteria 名稱 (3-8 字)
    description: string;  // 說明哪類 idea unit 屬於此分類 (供 embedding 比對用)
    raw_labels: string[]; // 被指派到此分類的所有 raw label
  }>
}
```

Output 目錄結構 (步驟 0-9 產出)

```
output_mock/
├─ mock_1.json           # Candidate (含 Step 1-9 輸出欄位)
├─ mock_2.json
├─ ...
├─ standard_dictionary.json # Step 3 產出
├─ embeddings_cache.json   # Step 3 embedding 快取
├─ clusters.json          # Step 7 分群結果摘要 (Step 8 補充命名與描述)
└─ umap.html              # Step 7 UMAP 互動式視覺化
```

限制事項

- 不得產生任何數值評分供展示
- LLM 呼叫不得在 prompt 中包含候選人姓名等識別資訊
- 所有分群邏輯基於向量距離與語意相似度，不依賴人工規則